AI Powered Trading Bot

Design Documentation

GROUP 8

Zayimhan Korkmaz - Mert Kedik - Ahmet Melih Bostancıeri

December, 2025

Course Project Submission

Contents

1 Introduction

1.1 Project Overview

This document outlines the design and architecture of the "AI Powered Trading Bot". The system is an automated trading platform designed to analyze real-time market data, execute trades based on technical analysis strategies, and manage a simulated wallet.

1.2 Problem Statement

Financial markets operate 24/7, making it impossible for human traders to monitor price movements continuously. Furthermore, manual calculation of complex indicators (like RSI, MACD) and executing trades with millisecond precision is prone to error and emotional bias.

1.3 Proposed Solution

We developed an Event-Driven Trading Bot using Spring Boot. The system utilizes a modular architecture allowing for dynamic strategy switching, real-time market observation, and automated execution. It supports both simulation and live market data integration.

1.4 Technology Stack

- **Backend:** Java, Spring Boot (Web, WebSocket)
- **Architecture:** MVC, Event-Driven
- **Data Handling:** WebSocket (Real-time), REST API
- **Tools:** Maven

1.5 Project Repository

The complete source code, including the Spring Boot backend, strategy implementations, and documentation, is hosted on GitHub.

GitHub Repo: https://github.com/zayimhan/tradebot

2 System Architecture

2.1 High-Level Architecture

The application follows the Model-View-Controller (MVC) pattern but is heavily enhanced with behavioral design patterns to handle the asynchronous nature of trading data.

2.2 Layer Descriptions

The project is structured into distinct layers to ensure separation of concerns:

Presentation Layer (Controller): Handles HTTP requests and interacts with the frontend.

Business Logic Layer (Service): Contains the core trading engine, wallet management, and market services.

Data Model Layer (Model): Defines the data structures for Market Data, Signals, and Trade records.

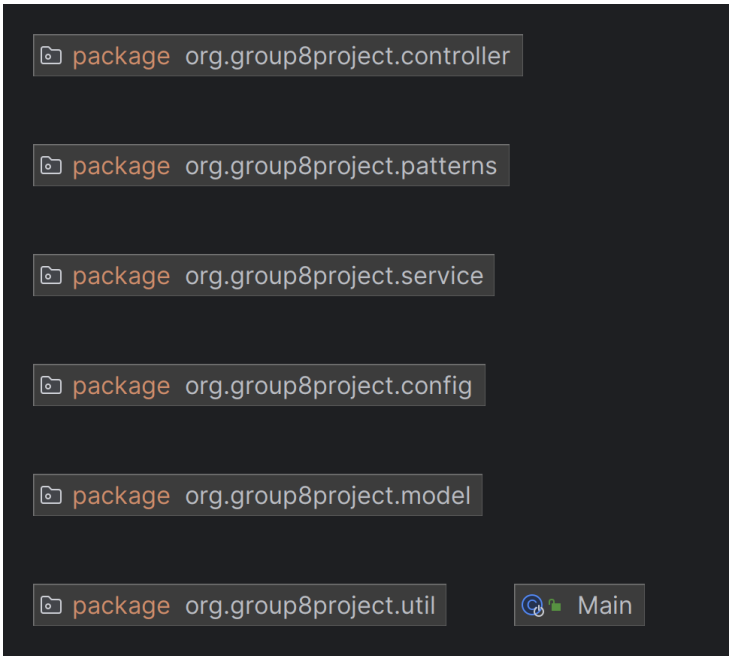

Figure 1: Project Package Structure

3 Design Patterns Implementation

3.1 Strategy Pattern

3.1.1 Pattern Overview

The Strategy pattern is used to define a family of trading algorithms, encapsulate each one, and make them interchangeable. This allows the `TradingEngine` to switch strategies (e.g., from RSI to MACD) at runtime without altering the engine's code.

3.1.2 Concrete Strategies

Table 1: Detailed Strategy Pattern Roles & Implementation

Pattern Role	**Class/Enum**	**Purpose & Responsibilities**
Strategy Interface	`TradingStrategy`	**The Contract.** Defines the rules all strategies must follow. Enforces implementation of `analyze(MarketData)`.
Context	`TradingEngine`	**The Executor.** Holds the `currentStrategy` reference and executes it. Keeps inactive strategies "warm" with background data.
Client (Director)	`AutoPilotService`	**The Manager.** Monitors volatility and switches the strategy on the `TradingEngine`. Acts as the intelligent driver.
Strategy Factory	`StrategyFactory`	**The Creator.** Creates and stores strategies in a Map. Handles conversion from String name (e.g., "RSI") to object instance.
Result Object	`TradeSignal` (Enum)	**The Output.** Represents standardized decisions: BUY, SELL, or HOLD. Ensures all strategies speak the same language.
Classification	`StrategyTerm` (Enum)	**The Label.** Defines timeframe (SCALPING, SHORT_TERM). Helps AutoPilot select the appropriate tool.
Concrete Strategy	`RSIStrategy`	**Tactic: Reversal.** Short-term strategy identifying overbought/oversold conditions based on RSI levels.
Concrete Strategy	`MacdStrategy`	**Tactic: Trend.** Mid-term strategy following trend changes using Moving Average Convergence Divergence.
Concrete Strategy	`MomentumStrategy`	**Tactic: Speed.** Focuses on the rate of price change to catch sudden breakouts.
Concrete Strategy	`BollingerBandsStrategy`	**Tactic: Volatility.** Monitors price deviations. Signals when price breaks out of bands or bands squeeze.
Concrete Strategy	`ScalpingStrategy`	**Tactic: High Freq.** Very fast strategy designed to profit from small price changes during high volatility.
Concrete Strategy	`GoldenCrossStrategy`	**Tactic: Major Trend.** Long-term strategy watching for intersection of 50-day and 200-day moving averages.

3.2 Factory Pattern

3.2.1 Pattern Overview and Rationale

The Factory Pattern is utilized to centralize the creation and retrieval of `TradingStrategy` objects. This design choice provides a key architectural benefit: the code that *uses* the strategies (`TradingEngine` and `BotController`) never knows how they are created. It only knows how to ask the factory for a strategy by its type name (e.g., "RSI").

3.2.2 Implementation: Dependency Injection as the Factory

Instead of manually initializing each strategy inside the factory, the system leverages **Spring's Dependency Injection (DI) Container** as a superior form of factory.

1. **Auto-Discovery:** Every concrete strategy class (like `RSIStrategy`) is annotated with `@Component`, signaling Spring to instantiate it as a bean.
2. **Auto-Injection:** The `StrategyFactory` is configured to accept a `List<TradingStrategy>` in its constructor. Spring automatically collects **all** available strategy implementations and injects them into the factory upon application startup.
3. **Map Storage:** The factory then registers these injected strategies into a private `strategyMap`, using their unique type string (e.g., "RSI", "MACD") as the key for quick lookup.

3.2.3 Critical Code Snippet (StrategyFactory.java)

The constructor method below is the core mechanism of the Factory, showcasing how the Strategy Pattern is integrated with the Spring ecosystem.

```
@Autowired
public StrategyFactory(List<TradingStrategy> strategies) {
    // Spring automatically injects ALL classes
    // implementing the TradingStrategy interface.

    for (TradingStrategy strategy : strategies) {
        String key = strategy.getStrategyType().toUpperCase();
        strategyMap.put(key, strategy);
        // ... logging ...
    }
    // ... logic for default "MA" alias ...
}

public TradingStrategy getStrategy(String type) {
    // Retrieval by name: the client (TradingEngine) only
    // interacts with the factory's public method.
    return strategyMap.get(type.toUpperCase());
}
```

3.2.4 Architectural Benefit: Open/Closed Principle (OCP)

The Factory Pattern ensures the system adheres strongly to the OCP:

- **Open for Extension:** To add a new trading strategy (e.g., `DemoStrategy`), a developer simply creates a new class implementing the `TradingStrategy` interface and annotates it with `@Component`.
- **Closed for Modification:** No existing code, including the `StrategyFactory` or the `TradingEngine`, needs to be modified. The new strategy is automatically discovered, registered, and made available to the entire system.

3.3 Observer Pattern

The Observer Pattern is implemented in two distinct areas of the application to ensure loose coupling between components.

3.3.1 Scenario 1: Market Data Flow (The Nervous System)

This implementation ensures that multiple components (Trading Engine, AutoPilot) can react to real-time price changes simultaneously without the Data Feed knowing who they are.

Table 2: Observer Pattern: Market Data Flow

Pattern Role	**Project Class**	**Purpose & Responsibilities**
Subject (Observable)	`MarketDataFeed`	**The Publisher.** Maintains the list of observers. When `publishPrice()` is called, it triggers the `update()` method for all subscribers. Serves as the single source of truth.
Observer Interface	`MarketObserver`	**The Contract.** Defines the `update(MarketData)` method. Ensures different classes (Engine, AutoPilot) communicate using a common interface.
Concrete Observer 1	`TradingEngine`	**The Execution Brain.** Implements MarketObserver. On every price update, it executes the active strategy (e.g., RSI) and triggers the `SpotExecutor` if a signal is generated.
Concrete Observer 2	`AutoPilotService`	**The Risk Manager.** Implements MarketObserver. Silently monitors price history to calculate volatility. Automatically switches the `TradingEngine`'s strategy (e.g., to "SCALPING") if market conditions change.

Pattern Role	Project Class	Purpose & Responsibilities
Client / Driver 1	`RealMarketService`	**Live Data Source.** Connects to Binance WebSocket. Pushes incoming real-time data into the system by calling `MarketDataFeed.publishPrice()`.
Client / Driver 2	`SimulationService`	**Test Data Source.** Generates synthetic market trends (Bull/Bear) and feeds simulated prices into `MarketDataFeed` for testing purposes.

3.3.2 Scenario 2: Wallet Notifications (Separation of Concerns)

This implementation separates the core business logic (managing money) from side effects (logging to file, updating UI).

Table 3: Observer Pattern: Wallet System

Pattern Role	Project Class	Purpose & Responsibilities
Subject (Observable)	`WalletService`	**The Vault.** Manages USD and Asset balances. When a `buy()` or `sellAll()` operation is successful, it calls `notifyObservers()` to broadcast the event.
Observer Interface	`WalletObserver`	**The Contract.** Defines the `onTrade()` method. Specifies what actions should be taken after a trade is completed.
Concrete Observer	`WalletLogger`	**The Recorder.** Implements WalletObserver. Listens for trade events and writes details to `wallet.log` and maintains an in-memory list for the UI.

3.4 Singleton Pattern

The Singleton Pattern is manually implemented in the `MarketDataFeed` class using the **Double-Checked Locking** mechanism. This ensures thread safety without the performance overhead of synchronizing the entire accessor method, and guarantees that only one instance handles the market data stream.

Table 4: Singleton Pattern Implementation (Double-Checked Locking)

Pattern Role	**Class / Concept**	**Implementation Details & Purpose**
Singleton Object	`MarketDataFeed`	**The Shared Instance.** Holds the list of observers and the latest price. The constructor is `private` to prevent external instantiation. It also includes a guard against Reflection attacks.
Lazy Initialization	`getInstance()`	**The Accessor.** The instance is created only when requested for the first time (Lazy Loading), not at startup. This optimizes resource usage.
Thread Safety	`Double-Checked Locking`	**Optimization.** Uses a `synchronized` block only when the instance is null. Once initialized, threads access the instance directly without locking overhead.
Memory Consistency	`volatile` Keyword	**Visibility.** Ensures that the `instance` variable is read directly from main memory, preventing instruction reordering issues in a multi-threaded environment.
Client Access	`Services & Engine`	**Global Access Point.** Classes like `TradingEngine` and `RealMarketService` access the feed via `MarketDataFeed.getInstance()` instead of Dependency Injection.

3.5 Template Method Pattern

This pattern is used to define the skeleton of the trade execution algorithm in a base class. It ensures that the critical sequence of a trade (Validation → Execution → Logging) is never violated, while allowing specific market types to implement their own logic.

Table 5: Template Method Pattern Implementation

Pattern Role	**Class / Method**	**Purpose & Responsibilities**
Abstract Class	`BaseTradeExecutor`	**The Skeleton.** Defines the high-level algorithm for executing a trade. It contains the template method and abstract declarations for steps that vary between markets.
Template Method	`executeTrade()`	**The Controller.** Declared as `final` to prevent overriding. It enforces a strict sequence: 1. Validate Funds → 2. Execute → 3. Log. It ensures no steps are skipped.

Pattern Role	Class / Method	Purpose & Responsibilities
Concrete Class	`SpotExecutor`	**Spot Implementation.** Implements the abstract steps for the Spot market. In `validateFunds()`, it specifically checks for real USD balance (for BUY) or Asset balance (for SELL) via `WalletService`.
Concrete Class	`FuturesExecutor`	**Futures Implementation.** Implements the steps for leveraged trading. Its `validateFunds()` logic is different (checks for "Initial Margin" instead of full asset amount) and it executes "Long/Short" positions.
Primitive Operation	`validateFunds()`	**Abstract Step.** A variable step defined in the parent but implemented by children. It forces every market type to define its own rules for "Do I have enough money?".
Primitive Operation	`performExecution()`	**Abstract Step.** The core action. `SpotExecutor` calls `walletService.buy()`, while `FuturesExecutor` would call an API for leveraged positions.
Concrete Operation	`logTransaction()`	**Common Step.** A shared method implemented directly in the base class. Since logging is the same for all trade types, it is not abstract, preventing code duplication.

4 UML Diagrams

4.1 Class Hierarchy Diagram

Below is the complete UML Class diagram representing the relationships between the Controller, Service, and Pattern layers.

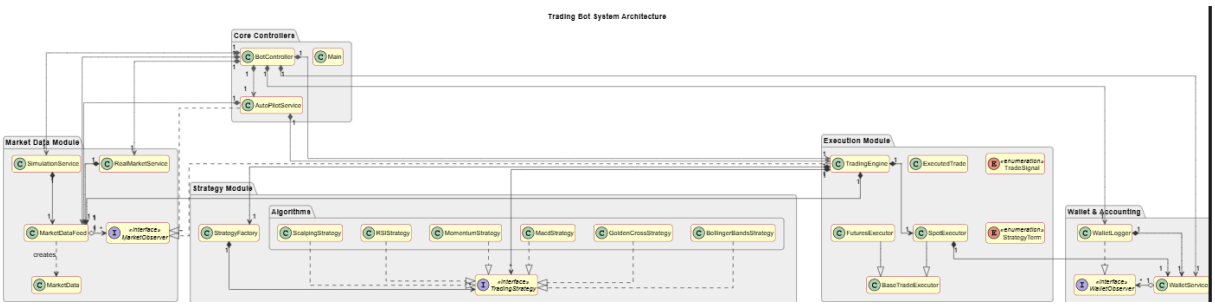

Figure 2: System Class Diagram

5 API Documentation

The bot exposes a comprehensive set of RESTful API endpoints for frontend control, monitoring, and debugging. The base URL for all endpoints is `/api/bot`.

Table 6: Complete REST API Reference

Method	**Endpoint**	**Parameters**	**Description**
GET	`/price`	-	Retrieves the current last known market price.
POST	`/price`	`symbol`, `price`	Manually pushes price data to the system (used for Simulation injection).
POST	`/simulation`	`active` (bool)	Starts or stops the internal automatic price simulation service.
POST	`/realmarket`	`active` (bool)	Toggles the connection to real-world market data (e.g., CoinGecko/Binance).
GET	`/realmarket/status`	-	Returns the boolean status of the real market connection.
GET	`/realmarket/connection`	-	**Debug:** Returns the current WebSocket connection state.
GET	`/klines`	`interval`, `limit`	Retrieves historical candlestick data for frontend charting (OHLCV format).
POST	`/strategy`	`type` (String)	Switches the active trading strategy (e.g., "RSI", "MACD", "SCALPING").
POST	`/autopilot`	`enable` (bool)	Toggles the AI-powered AutoPilot mode on or off.
GET	`/balance`	-	Returns the current available USD balance in the wallet.

Method	Endpoint	Parameters	Description
GET	`/assets`	`symbol`	Returns the held amount of a specific asset (e.g., BTC).
GET	`/wallet-logs`	-	Returns the full history of executed trades and wallet events in JSON format.

6 Future Work & Conclusion

6.1 Future Improvements

While the current system provides a robust foundation for algorithmic trading, several enhancements are planned to transition from a term project to a production-ready application:

- **Integration of Deep Learning Models:** Currently, the "AutoPilot" makes decisions based on statistical volatility. The next step is to integrate a **Python-based Microservice** (using Flask or FastAPI) running LSTM or Transformer models to predict price trends, communicating with our Java backend via REST.
- **Multi-Channel Notification System:** Leveraging the existing `WalletObserver` infrastructure, we plan to implement new observers for **Telegram** and **Email** notifications. Thanks to the loose coupling in our design, this can be achieved without modifying the core `WalletService`.
- **Backtesting Engine:** Before risking real capital, a backtesting module will be added. This will allow us to replay historical market data through the `TradingEngine` to evaluate the performance of strategies like MACD and RSI over past periods.

6.2 Conclusion

The primary challenge of this project was not just implementing trading logic, but managing the complexity of real-time asynchronous data flow. By strictly adhering to software design patterns, we avoided the common pitfall of "spaghetti code" often seen in event-driven systems.

- The **Observer Pattern** successfully decoupled the market data feed from the trading logic, allowing the system to scale.
- The **Strategy and Factory Patterns** proved that the application complies with the **Open/Closed Principle**; we were able to add new strategies during development without breaking existing functionality.

In conclusion, this project demonstrated that a well-architected system using Spring Boot and proper design patterns significantly reduces technical debt and provides a flexible infrastructure for future financial technologies.